



Apex Institute of Technology

Computer Science & Engineering

Experiment 8

Name: Shivam Abhepal

UID: 24BAI70214

Branch: B.E. CSE (AIML)

Section: 24AIT_KRG-G1

Semester: 4

Date of Performance: 27.03.2026

Subject Name: Database
Management System

Subject Code: 24CSH-298

AIM: This experiment focuses on designing and implementing a stored procedure in PostgreSQL. It demonstrates how to use IN, OUT, and INOUT parameters to process and return data dynamically.

OBJECTIVES:

- To understand stored procedures in PostgreSQL
- To learn the use of IN, OUT, and INOUT parameters
- To execute procedures using DO block
- To retrieve and display processed results

SOFTWARE REQUIREMENTS:

- PostgreSQL (pgAdmin / DB Fiddle)
- SQL (DDL, DML)
- PL/pgSQL

PROCEDURE/STEPS:

1. Start the system and log in
2. Open pgAdmin / DB Fiddle
3. Connect to the database
4. Open Query Tool



Apex Institute of Technology

Computer Science & Engineering

5. Create employees table
6. Insert sample data
7. Create stored procedure
8. Execute procedure using DO block
9. Observe output

PRACTICAL/EXPERIMENTAL STEPS:

```
DROP TABLE IF EXISTS employees;
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    emp_name VARCHAR(50),
    gender VARCHAR(10),
    salary NUMERIC(10,2)
);
INSERT INTO employees (emp_name, gender, salary) VALUES
('Anhad', 'Male', 30000),
('Riya', 'Female', 35000),
('Jaskaran', 'Male', 28000),
('Simran', 'Female', 40000),
('Yuvraj', 'Male', 32000);
```

-- Stored Procedure

```
CREATE OR REPLACE PROCEDURE get_Employee_Count_BY_Gender (
    IN IN_GENDER VARCHAR(20),
    OUT OUT_COUNT INT,
    INOUT STATUS VARCHAR(20)
)
AS
$$
BEGIN
    SELECT COUNT(*)
    INTO OUT_COUNT
    FROM employees
```



Apex Institute of Technology

Computer Science & Engineering

```
WHERE GENDER = 'Male';
```

```
STATUS := 'SUCCESS';
```

```
END;
```

```
$$ LANGUAGE PLPGSQL;
```

```
-----  
-- Execution Block  
-----
```

```
DO
```

```
$$
```

```
DECLARE
```

```
    GEN VARCHAR(20) := 'Male';
```

```
    Count_of_Employee INT;
```

```
    STATUS VARCHAR := 'Pending';
```

```
BEGIN
```

```
    CALL get_Employee_Count_BY_Gender(GEN, Count_of_Employee, STATUS);
```

```
    RAISE NOTICE
```

```
    'YOUR COUNT OF EMPLOYEE BY GENDER IS % AND YOUR STATUS IS %',
```

```
    Count_of_Employee, STATUS;
```

```
END;
```

```
$$;
```



Apex Institute of Technology

Computer Science & Engineering

OUTPUT:(Screenshots)

```
29 SELECT COUNT(*)
30 INTO out_count
31 FROM employees
32 WHERE LOWER(gender) = LOWER(in_gender);
33
34 status := 'SUCCESS';
35
36 END;
37 $$
38
39 -- Execution block
40 DO $$
41 DECLARE
42     gen VARCHAR(20) := 'Male';
43     count_of_employee INT;
44     status VARCHAR(20) := 'Pending';
45 BEGIN
46     CALL get_employee_count_by_gender(gen, count_of_employee, status);
47     RAISE NOTICE 'YOUR COUNT OF EMPLOYEE BY GENDER IS % AND YOUR STATUS IS %',
48         count_of_employee, status;
49 END;
50 $$
```

Data Output Messages Notifications

NOTICE: YOUR COUNT OF EMPLOYEE BY GENDER IS 3 AND YOUR STATUS IS SUCCESS
DO

Query returned successfully in 42 msec.

I/O ANALYSIS:

Input:

- Employee Name
- Gender
- Salary

Output:

- Count of employees by gender
- Execution status



Apex Institute of Technology

Computer Science & Engineering

LEARNING OUTCOMES:

- Understood stored procedures in PostgreSQL
- Learned usage of IN, OUT, and INOUT parameters
- Executed procedures using DO block
- Displayed output using RAISE NOTICE
- Gained hands-on experience with PL/pgSQL

RESULT:

The stored procedure successfully calculates the number of employees and returns the result along with execution status.

CONCLUSION:

Stored procedures in PostgreSQL allow efficient data processing using parameters and reusable logic.